AI Course Project Abstract

Rachel Bell, Allison Shay O'Brien, Emily Tew

October 21, 2026

Abstract

Chess has far too many possible move sequences to search by brute force. In this project, we will train an agent, Blobfish, to choose strong moves by combining Monte Carlo Tree Search (MCTS) with Actor-Critic reinforcement learning: a neural network suggests promising moves and estimates how good positions are, and MCTS uses those estimates to search selectively. The agent will learn entirely through self-play, with the `python-chess` library handling the rules of the game, and we will measure its progress by playing it against Stockfish at increasing difficulty levels.

1 Project Title: Blobfish

In a game like chess, there are too many possible move sequences to evaluate using brute force. Our project addresses this by training an agent to identify promising moves and evaluate positions, allowing it to search more selectively and improve through self-play. Ultimately, given the current state of the chess board, our agent aims to choose the move with the highest likelihood of winning.

Since the rules of chess are fully known and games can be simulated, we will not need any traditional datasets. Instead, the agent will play against itself in actual games of chess. Since we will be going through a very large number of chess states, we will use the python-chess library rather than trying to make a performant chess engine ourselves, and pre-existing chess AIs like Stockfish to test our own (see Section 3).

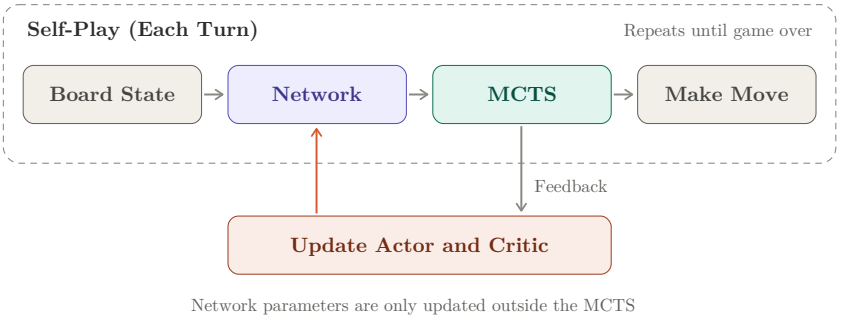
Table 1 below summarizes our current problem formulation. This may change slightly as we develop the agent, but the general ideas should stay the same.

Component	**Description**
State s_t	The current board state at time t
Action a_t	A valid move from the current player's perspective at time t
Policy $\pi_{\boldsymbol{\theta}}$	The output of the actor part of the network with parameters $\boldsymbol{\theta}$
Transition $T(s_t, a_t)$	Deterministic: any action always leads to exactly one next state s_{t+1}
Reward $R(s_t, p)$	The goodness of the current state relative to player p

Table 1: Components of the problem formulation

2 Algorithm Implementation and Development

The primary methods we will be using for our project will be Monte Carlo Tree Search (MCTS) and Actor-Critic Reinforcement Learning. This general approach, a learned network guiding MCTS and improving through self-play, is similar to AlphaZero [1], though at a much smaller scale. The figure below gives an overview of how the two fit together during each turn in self-play.



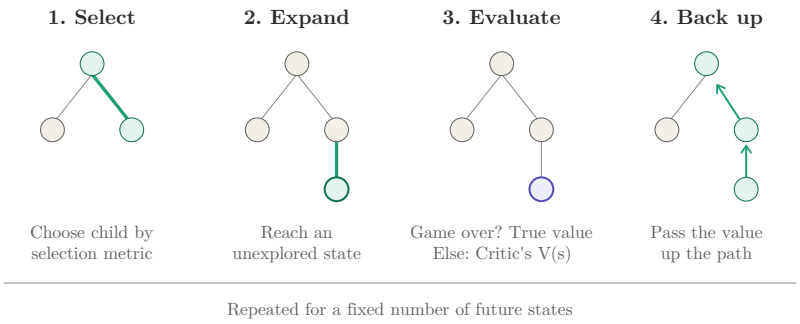
2.1 Reward

Ultimately, the value of a chess state comes from finished games, where V can be, for example, -1 for a loss, 0 for a draw, or 1 for a win, relative to the player. Since this reward is very sparse (only occurring at the end of a game), and both good and bad actions can lead to a win or a loss, we plan to use MCTS to give the agent feedback. Traditionally, MCTS uses the UCB1 (Upper Confidence Bound) formula [2, 3] to determine which child node to visit next.

$$UCB1(i) = \bar{X}_i + c\sqrt{\frac{\ln N}{n_i}} \tag{1}$$

Here, $\bar{X}_i$ is the average value of child i, n_i is the number of times it has been visited, N is the number of times its parent has been visited, and c is a constant that balances exploring less-visited moves against exploiting moves that already look good. UCB1 will be our starting point for choosing which nodes to explore, but since we plan to test variations, we refer to it more generally as the *selection metric* in the figures and Algorithm 1. For example, incorporating the agent's move probabilities could steer the search in a more beneficial direction. In any case, MCTS will explore a fixed number of future states to estimate how good the current state is. Unlike traditional MCTS, we will evaluate unexplored states with the critic's value estimate (see Section 2.2) instead of a full rollout (playing random or agent-chosen moves until the game ends). This saves substantial computation, since running the network for every move of every rollout in every search quickly becomes expensive, and random rollouts are poorly suited to chess. Once the agent is skilled at estimating state values, this approximation should be fairly accurate. After the search, the move actually played is selected by MCTS rather than sampled directly from the actor's output.

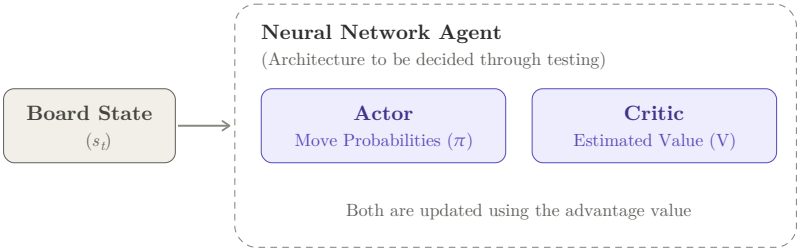
At any point during the MCTS, if a true terminal state is reached (i.e. the game is over), the actual value of that state will be used instead of the critic's prediction. Without this, the agent would not be receiving any ground-truth feedback. Put together, the agent will ideally learn similar to how a real person would: by playing games, seeing which end favorably and which do not, and slowly adapting to win more often. The figure below illustrates one iteration of this search:



This is the core of how the agent will receive feedback, but we would also like to experiment with variations on MCTS. For example, hybrid MCTS and Minimax algorithms have been proposed, since MCTS might miss moves that initially look bad but lead to favorable states (a great example in chess is sacrificing a piece to gain an advantage later on). To help with that, we also want to look into simple heuristic functions (such as piece advantage), so that we can search for states with a large advantage, not just winning terminal states. Of course, any heuristic we could realistically make will inevitably score some bad states highly, so it would need to be balanced with the rest of the MCTS rather than prioritized over all else. Even if these ideas do not increase the agent's best performance, they might lower the number of games it needs to reach that point.

2.2 Reinforcement Learning

Currently, our plan is to use Actor-Critic reinforcement learning [4]. The agent will consist of two parts: the actor, which outputs the policy π (probabilities for taking each action), and the critic, which outputs the estimated value V of the current state.



Both will be updated using the advantage value, essentially a measure of how much better the new state is than the current one. This way, the agent is not rewarded for making bad moves while already in good positions (something which can happen often in chess), but must continually make moves that lead to good states. Below, Algorithm 1 gives a simple pseudo implementation of the full training process, which we plan to adjust as needed.

Algorithm 1: Blobfish self-play training with MCTS and Actor-Critic

Input: actor $\pi(a|s, \boldsymbol{\theta})$, critic $\hat{v}(s, \mathbf{w})$, number of MCTS simulations K
Repeat for each self-play game:
  Initialize S (starting board state)
  While S is not terminal:
    Get move probabilities $\pi(\cdot|S, \boldsymbol{\theta})$ from the actor
    Repeat K times (network parameters fixed):
      Select: descend the tree using a selection metric
      Expand: reach an unexplored state S'
      Evaluate: $v \leftarrow$ true game result if S' is terminal, else $v \leftarrow \hat{v}(S', \mathbf{w})$
      Back up: propagate v up the search path
    $A \leftarrow$ move selected by MCTS
    Compute the advantage from MCTS feedback and $\hat{v}(S, \mathbf{w})$
    Update $\mathbf{w}$ (critic) and $\boldsymbol{\theta}$ (actor) using the advantage
    $S \leftarrow T(S, A)$

When it comes to the game itself, the agent will learn by playing only against itself (anything else will be for testing alone). This ensures it is always playing an opponent of a similar skill level. With two separate agents, if one became significantly better than the other, the weaker one would have a much harder time learning: if positive ground-truth feedback only comes from winning, an agent that's significantly worse than its opponent will rarely, if ever, see it. While self-play may lead to problems further down the road, we will hopefully be able to deal with them.

2.3 Neural Network

We plan to use a neural network to accomplish this implementation, as chess has far, far too many states for many other methods to be feasible. Our current ideas for the architecture include a ResNet-style network and a transformer-based network, but we hope to test several different architectures to find what works well. Of note, the network's parameters will only be updated outside of the MCTS, as including the critic's usage in the search in the gradient calculation would likely take a very, very long time.

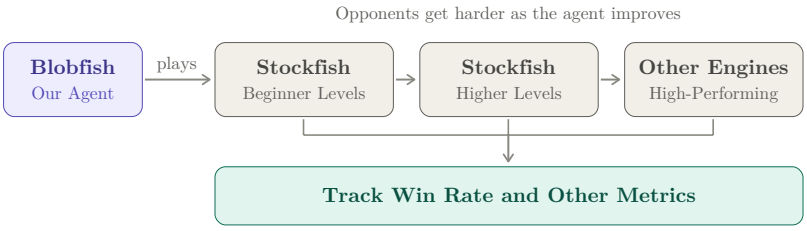
2.4 Python Libraries

We will use the `python-chess` library to handle the rules and representation of the game, so we can focus on the agent itself rather than on implementing and debugging the rules of chess. It provides:

- **Legal move generation and validation** that is both fast and correct. This matters because MCTS and self-play evaluate an enormous number of board states, and any bug in the game logic would feed incorrect states into training.
- **Game-ending detection**, including checkmate, stalemate, and draws by repetition or the fifty-move rule.
- **An engine module** that communicates with UCI-compatible engines such as Stockfish, letting us play our agent against it at different difficulty levels (see Section 3).

3 Measuring Improvement

We will likely start with Stockfish, the most famous chess engine, as a baseline. Its built-in difficulty settings let us begin at beginner strength while our agent is still training, then raise the difficulty and switch to other high-performing engines as the agent improves. We will track progress through win, draw, and loss rates (along with other metrics) against each opponent, as shown below.



4 Summary and Conclusions

Our goal for Blobfish is to train a chess-playing agent through self-play by combining Monte Carlo Tree Search with Actor-Critic reinforcement learning. Rather than relying on brute-force search, the agent will learn to estimate the value of board states and focus its search on promising moves. We will use the python-chess library for game logic and measure progress against Stockfish at increasing difficulty levels.

To divide the work, each team member will take primary ownership of one major component:

- **Allison** - Monte Carlo Tree Search, including experimenting with hybrid MCTS/Minimax and heuristic evaluation.
- **Emily** - Actor-Critic learning and the neural network, including testing ResNet-style and transformer-based architectures.
- **Rachel** - Game environment and evaluation, including python-chess integration, the self-play loop, and benchmarking against Stockfish.

These roles define who leads each area, not hard boundaries. Since the components depend heavily on one another, we expect everyone to contribute across all parts of the project.

References

[1] D. Silver *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

[2] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. European Conference on Machine Learning (ECML)*, 2006, pp. 282–293.

[3] GeeksforGeeks, "Monte Carlo Tree Search (MCTS) in Machine Learning," GeeksforGeeks, May 2026. [Online]. Available: `https://www.geeksforgeeks.org/machine-learning/monte-carlo-tree-search-mcts-in-machine-learning/` (accessed Oct. 21, 2026).

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.